

torch.linalg.lu#

<https://pytorch.org/docs/stable/generated/torch.linalg.lu.html>

Description:

Computes the LU decomposition with partial pivoting of a matrix. Letting $K \in \mathbb{R}^{m \times n}$ or $\mathbb{C}^{m \times n}$, the LU decomposition with partial pivoting of a matrix $A \in K^{m \times n}$ is defined as where $k = \min(m, n)$, PPP is a permutation matrix, LLL is lower triangular with ones on the diagonal and UUU is upper triangular. If `pivot= False` and A is on GPU, then the LU decomposition without pivoting is computed. When `pivot= False`, the returned matrix P will be empty. The LU decomposition without pivoting may not exist if any of the principal minors of A is singular. In this case, the output matrix may contain `inf` or `NaN`.

Function Signature

```
torch.linalg.lu(A, *, pivot=True, out=None)#
```

Parameters

Keyword Arguments

Returns

Parameters

Keyword

out (tuple, optional) – output tuple of three tensors. Ignored if None. Default: None.

Returns:

A named tuple (P, L, U).

Code Examples

```
>>> A = torch.randn(3, 2)
>>> P, L, U = torch.linalg.lu(A)
>>> P
tensor([[0., 1., 0.],
        [0., 0., 1.],
        [1., 0., 0.]])
>>> L
tensor([[1.0000, 0.0000],
        [0.5007, 1.0000],
        [0.0633, 0.9755]])
>>> U
tensor([[0.3771, 0.0489],
        [0.0000, 0.9644]])
>>> torch.dist(A, P @ L @ U)
tensor(5.9605e-08)

>>> A = torch.randn(2, 5, 7, device="cuda")
>>> P, L, U = torch.linalg.lu(A, pivot=False)
>>> P
tensor([], device='cuda:0')
>>> torch.dist(A, L @ U)
tensor(1.0376e-06, device='cuda:0')
```

Notes & Warnings

NOTE

See also `torch.linalg.solve()` solves a system of linear equations using the LU decomposition with partial pivoting.

⚠ WARNING

Warning The LU decomposition is almost never unique, as often there are different permutation matrices that can yield different LU decompositions. As such, different platforms, like SciPy, or inputs on different devices, may produce different valid decompositions.

⚠ WARNING

Warning Gradient computations are only supported if the input matrix is full-rank. If this condition is not met, no error will be thrown, but the gradient may not be finite. This is because the LU decomposition with pivoting is not differentiable at these points.

Detailed Documentation

Documentation

Computes the LU decomposition with partial pivoting of a matrix.

Letting $K \in \{\mathbb{R}, \mathbb{C}\}$, the LU decomposition with partial pivoting of a matrix $A \in K^{m \times n}$ is defined as

where $k = \min(m, n)$, PPP is a permutation matrix, LLL is lower triangular with ones on the diagonal and UUU is upper triangular.

If `pivot= False` and A is on GPU, then the LU decomposition without pivoting is computed

When `pivot= False`, the returned matrix P will be empty. The LU decomposition without pivoting may not exist if any of the principal minors of A is singular. In this case, the output matrix may contain inf or NaN.

Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if A is a batch of matrices then the output has the same batch dimensions.

`torch.linalg.solve()` solves a system of linear equations using the LU decomposition with partial pivoting.

The LU decomposition is almost never unique, as often there are different permutation matrices that can yield different LU decompositions. As such, different platforms, like SciPy, or inputs on different devices, may produce different valid decompositions.

Gradient computations are only supported if the input matrix is full-rank. If this condition is not met, no error will be thrown, but the gradient may not be finite. This is because the LU decomposition with pivoting is not differentiable at these points.

A (Tensor) – tensor of shape $(*, m, n)$ where $*$ is zero or more batch dimensions.

pivot (bool, optional) – Controls whether to compute the LU decomposition with partial pivoting or no pivoting. Default: True.

out (tuple, optional) – output tuple of three tensors. Ignored if None. Default: None.

A named tuple (P, L, U).